Aubrée Romain **Beauvallet**
07/11/2026 **Semestre 3**
SLAM

Groupe 1

Bloc 2 TP 1 React

Contexte **2**
Partie 1 : Installation de l'environnement **2**
Partie 2 : Création du projet **3**
Partie 3 : Découverte de l'arborescence **4**
Partie 4 : Premier composant en JSX **5**
Partie 5 : Les props **7**
Partie 6 : L'état avec useState **8**
Partie 7 : Afficher une liste **9**
Pour aller plus loin **11**
Conclusion **12**

Contexte

Ce TP a pour but de découvrir **React** à travers la réalisation d'un premier prototype : une petite application de suivi de tickets pour l'entreprise WebDev Solutions. On commence par installer l'environnement (**Node.js** et **Vite**), on crée un projet, puis on apprend à écrire des composants en **JSX**, à utiliser les **props** et le **state**, et enfin à afficher une liste dynamique de données.

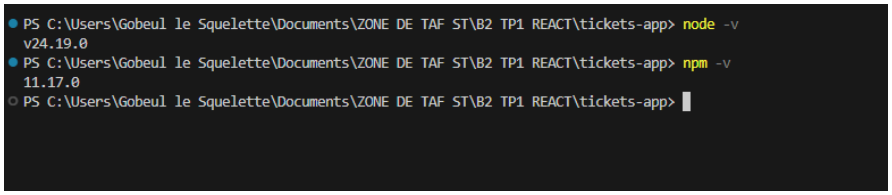
*Le projet est disponible sur **GitHub** (**https://github.com/RomAub/bvl_S3_B2_TP1_React**)*

Partie 1 : Installation de l'environnement

React s'appuie sur **Node.js** et son gestionnaire de paquets **npm**. J'installe donc la version **LTS** de Node.js depuis le site officiel (nodejs.org), puis je vérifie que tout est bien installé dans un terminal :

Q1 Quelles versions de Node et npm sont installées ? À quoi sert npm ?

Avec les commandes **node -v** et **npm -v**, j'obtiens **Node v24.19.0** et **npm 11.17.0**.



npm (**Node Package Manager**) est le gestionnaire de paquets de Node. Il sert à installer, mettre à jour et gérer les bibliothèques (les dépendances) d'un projet, comme React, et à lancer les scripts définis dans le fichier **package.json**.

Pour être plus à l'aise, j'installe aussi trois extensions dans VS Code : **ES7+ React/Redux/React-Native snippets** (raccourcis de code React), **Prettier** (formatage automatique du code) et **ESLint** (détection des erreurs).

Partie 2 : Création du projet

Je crée le projet avec Vite, puis je l'installe et je le lance, à l'aide de quatre commandes :

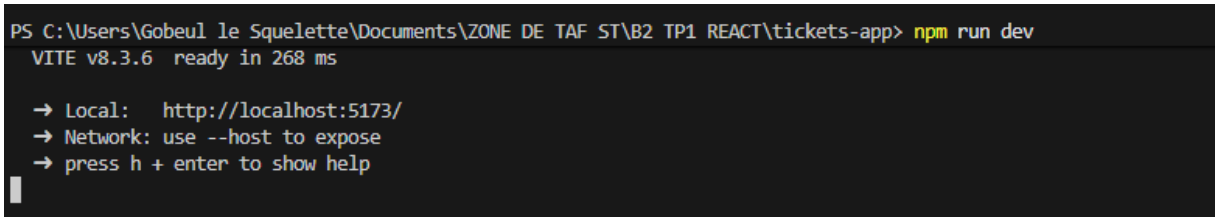
Q2 Que fait chacune des quatre commandes ?

npm create vite@latest tickets-app -- --template react : crée un nouveau projet nommé tickets-app à partir du modèle React de Vite.

cd tickets-app : entre dans le dossier du projet.

npm install : télécharge toutes les dépendances listées dans le package.json (elles sont placées dans le dossier node_modules).

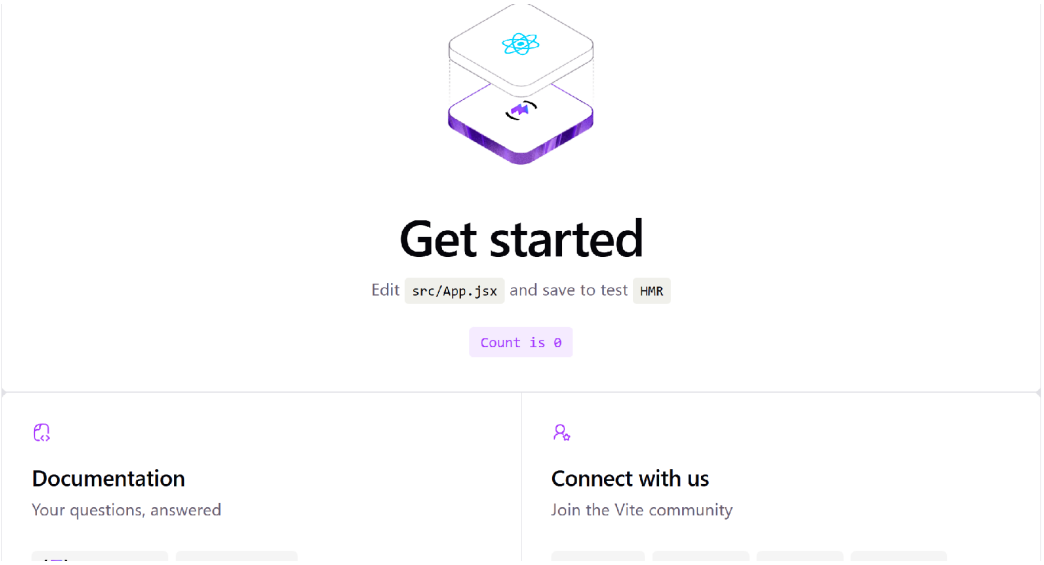
npm run dev : lance le serveur de développement de Vite ; l'application devient alors accessible dans le navigateur à l'adresse indiquée.



Q3 Qu'est-ce que Vite et quel est son rôle ? (le comparer à un serveur web classique)

Vite est un outil de build et un serveur de développement pour les applications web modernes. Il sert à démarrer le projet très rapidement et à recharger automatiquement la page à chaque modification du code (**Hot Module Replacement**). Contrairement à un serveur web classique comme Apache (dans XAMPP), qui se contente de servir des fichiers déjà prêts, Vite transforme le code à la volée pendant le développement et ne reconstruit que ce qui a changé, ce qui le rend beaucoup plus rapide.

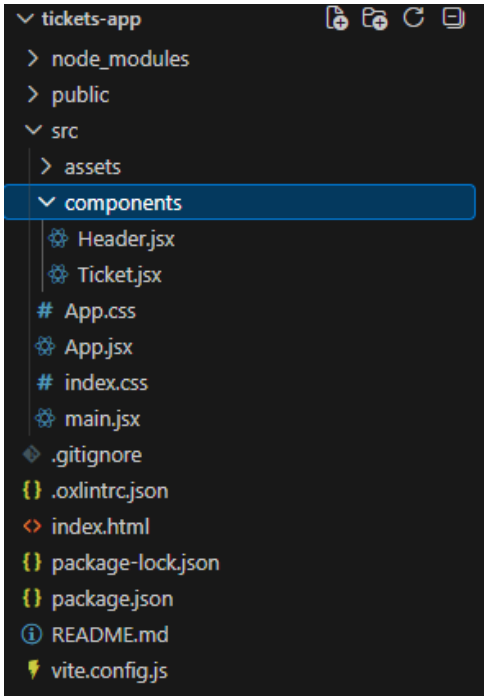
Une fois le serveur lancé, j'ouvre l'adresse **http://localhost:5173** dans le navigateur et j'obtiens la page d'accueil par défaut de Vite + React :



Partie 3 : Découverte de l'arborescence

Vite a généré une arborescence de projet. Voici le rôle des principaux fichiers et dossiers :

Fichier / dossier	Rôle
package.json	Informations du projet, dépendances et scripts (dev, build…).
node_modules/	Contient toutes les dépendances installées.
index.html	Page HTML de base ; elle contient la div #root où React s'affiche.
src/main.jsx	Point d'entrée : il « monte » le composant App dans la page.
src/App.jsx	Composant principal de l'application.
src/assets/	Ressources du projet (images, icônes…).



Q4 Dans main.jsx, que fait la ligne createRoot(document.getElementById('root')).render(...) ?

createRoot(document.getElementById('root')) récupère la balise ** ** du fichier index.html et en fait la racine de l'application React. La méthode **.render(...)** y affiche ensuite le composant passé en paramètre (ici **<App />**). C'est donc cette ligne qui relie le code React à la page HTML.

Partie 4 : Premier composant en JSX

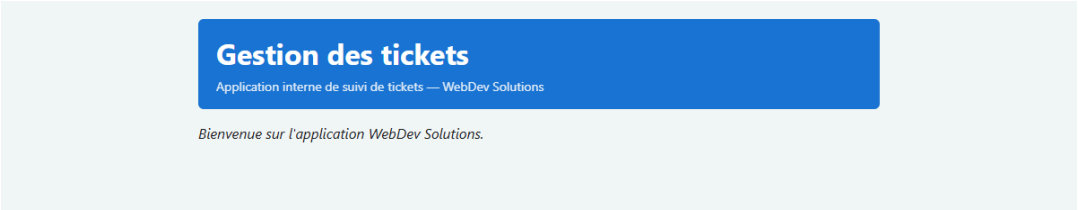
Je remplace le contenu de **App.jsx** par un affichage simple, je vide les fichiers CSS, puis je crée un dossier **src/components/** avec un premier composant **Header.jsx** que j'importe dans App :

```jsx
function Header() {
  return (
    <header className="header">
      <h1>Gestion des tickets</h1>
      <p>Application interne de suivi de tickets — WebDev Solutions</p>
    </header>
  );
}

export default Header;

```

Dans le navigateur, le composant **Header** s'affiche bien en haut de la page :



Q5 Qu'est-ce que le JSX ? Citez trois différences avec le HTML (ex. class / className).

Le **JSX** (JavaScript XML) est une syntaxe qui permet d'écrire du code ressemblant à du HTML directement dans du JavaScript. Trois différences avec le HTML : on écrit **className** au lieu de **class** ; les attributs sont en camelCase (ex. **onClick** et non onclick) ; et on peut insérer des variables ou des expressions JavaScript à l'aide d'accolades **{ }**.

Q6 Pourquoi un composant doit-il retourner un seul élément parent ?

Parce que la fonction d'un composant ne peut retourner qu'une seule valeur. Il faut donc envelopper tout le contenu dans un seul élément parent (par exemple une ** ** ou un fragment **<>…</>**), sinon React renvoie une erreur.

Partie 5 : Les props

Je crée un composant **Ticket.jsx** qui reçoit des **props** (titre, auteur, priorité et statut) et les affiche. J'ajoute une prop **statut** affichée en rouge si « Ouvert », orange si « En cours » et vert si « Résolu » grâce à un rendu conditionnel :

```jsx
function couleurStatut(statut) {
  if (statut === "Ouvert") return "#d32f2f";
  if (statut === "En cours") return "#ed6c02";
  return "#2e7d32";
}

function Ticket({ titre, auteur, priorite, statut }) {
  return (
    <div className="ticket">
      <h3>{titre}</h3>
      <p>Déclaré par : {auteur}</p>
      <p>Priorité : {priorite}</p>
      <p>
        Statut :{" "}
        <span className="statut" style={{ color: couleurStatut(statut) }}>
          {statut}
        </span>
      </p>
    </div>
  );
}

export default Ticket;
```

J'utilise ensuite ce composant dans App avec plusieurs tickets différents. Chaque statut s'affiche bien avec la bonne couleur :

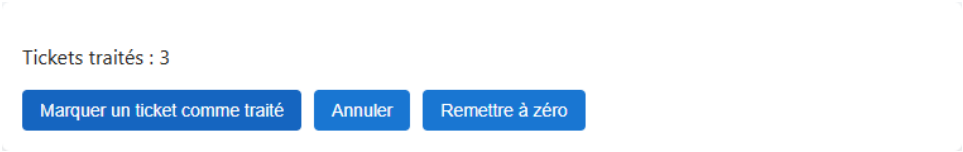


Q7 Les props sont-elles modifiables par le composant qui les reçoit ? Justifiez.

Non. Les props sont en **lecture seule** : le composant qui les reçoit ne doit pas les modifier. Elles sont transmises par le composant parent et servent uniquement à afficher des données. Si une valeur doit changer dans le temps, on utilise plutôt le **state**.

Partie 6 : L'état avec useState

J'ajoute un compteur de tickets traités dans App grâce à **useState**. Un bouton incrémente le compteur, un autre le remet à zéro, et un bouton « Annuler » l'empêche de descendre sous 0 :



Q8 Quelle est la différence entre une prop et un state ?

Une **prop** est une donnée transmise par le composant parent : elle est fixe du point de vue de l'enfant (lecture seule). Un **state** est une donnée gérée à l'intérieur du composant lui-même : elle peut changer (avec sa fonction **set…**) et, quand elle change, React réaffiche automatiquement le composant.

Q9 Pourquoi ne faut-il pas écrire traites = traites + 1 directement ?

Parce que React ne verrait pas le changement : modifier directement la variable ne déclenche pas de nouveau rendu, donc l'affichage ne se mettrait pas à jour. Il faut passer par la fonction **setTraites(...)**, qui met à jour le state et demande à React de réafficher le composant.

Partie 7 : Afficher une liste

Au lieu d'écrire les trois **<Ticket />** à la main, je place les données dans un tableau et je les affiche avec **map()**, en donnant une prop **key** unique à chaque élément. Voici le composant App :

```jsx
import { useState } from "react";
import Header from "./components/Header";
import Ticket from "./components/Ticket";
import "./App.css";

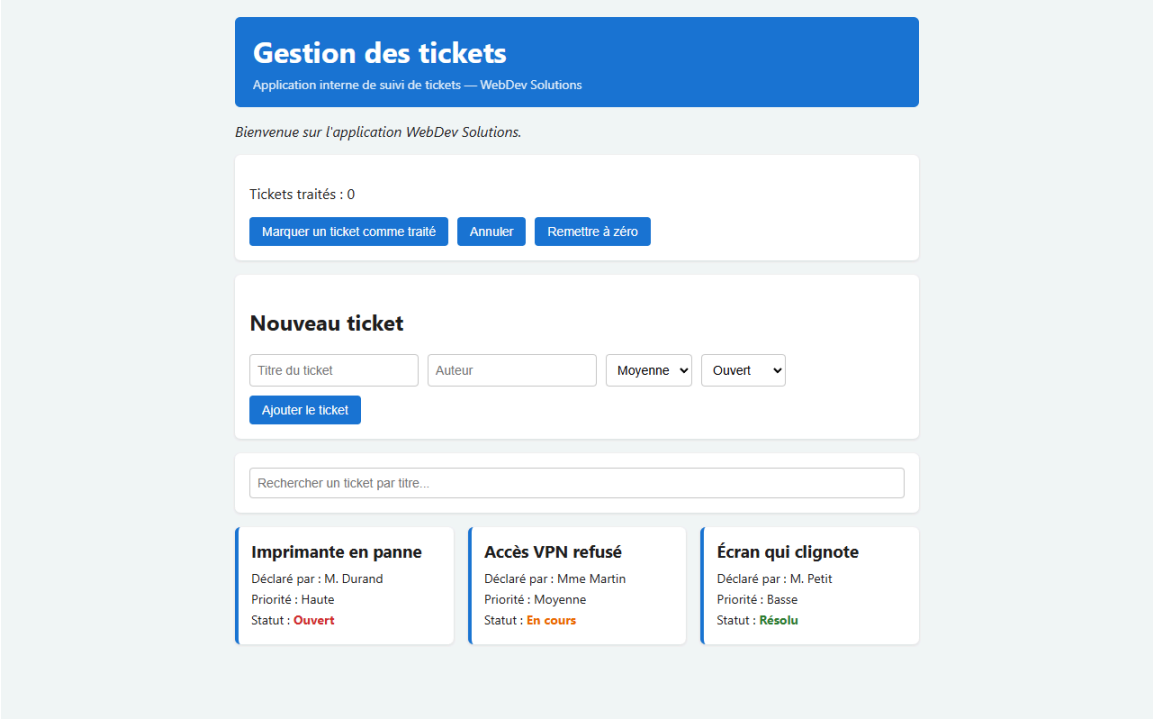
const ticketsInitiaux = [
  { id: 1, titre: "Imprimante en panne", auteur: "M. Durand", priorite: "Haute", statut: "Ouvert" },
  { id: 2, titre: "Accès VPN refusé", auteur: "Mme Martin", priorite: "Moyenne", statut: "En cours" },
  { id: 3, titre: "Écran qui clignote", auteur: "M. Petit", priorite: "Basse", statut: "Résolu" },
];

function App() {
  const [traites, setTraites] = useState(0);
  const [recherche, setRecherche] = useState("");
  const [tickets, setTickets] = useState(ticketsInitiaux);
  const [titre, setTitre] = useState("");
  const [auteur, setAuteur] = useState("");
  const [priorite, setPriorite] = useState("Moyenne");
  const [statut, setStatut] = useState("Ouvert");

  const ajouterTicket = (e) => {
    e.preventDefault();
    if (titre.trim() === "" || auteur.trim() === "") return;
    const nouveau = { id: Date.now(), titre, auteur, priorite, statut };
    setTickets([...tickets, nouveau]);
    setTitre("");
    setAuteur("");
    setPriorite("Moyenne");
    setStatut("Ouvert");
  };

  const ticketsFiltres = tickets.filter((t) =>
    t.titre.toLowerCase().includes(recherche.toLowerCase())
  );
```

L’application affiche bien la liste des tickets générée à partir du tableau :

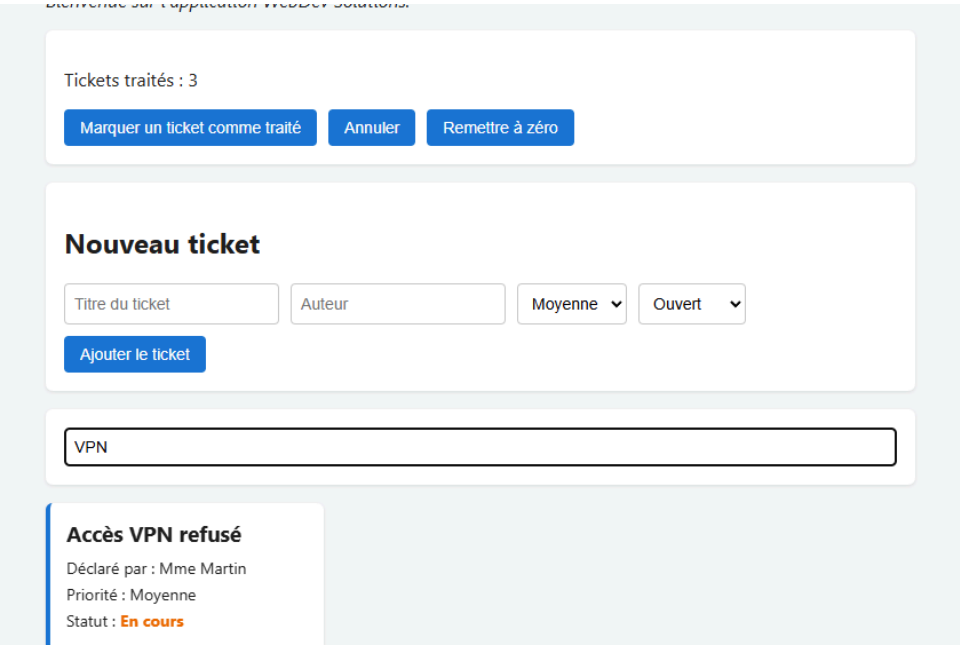


<u>Q10</u> À quoi sert la prop key ? Que se passe-t-il si on l’oublie (console du navigateur) ?

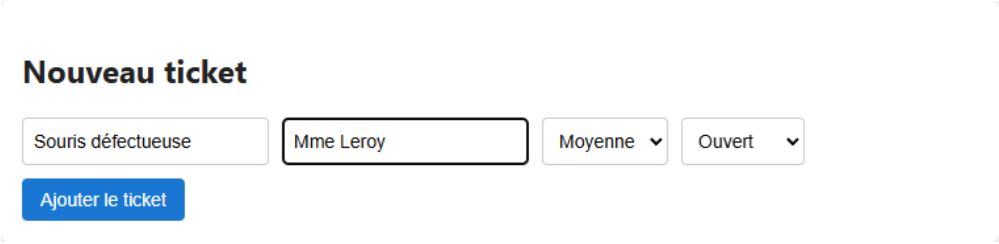
La prop **key** donne à chaque élément d’une liste un identifiant unique. Elle permet à React de reconnaître les éléments et de mettre à jour la liste efficacement (ajout, suppression…). Si on l’oublie, React affiche un avertissement dans la console du navigateur : **« Each child in a list should have a unique "key" prop »**.

Pour aller plus loin

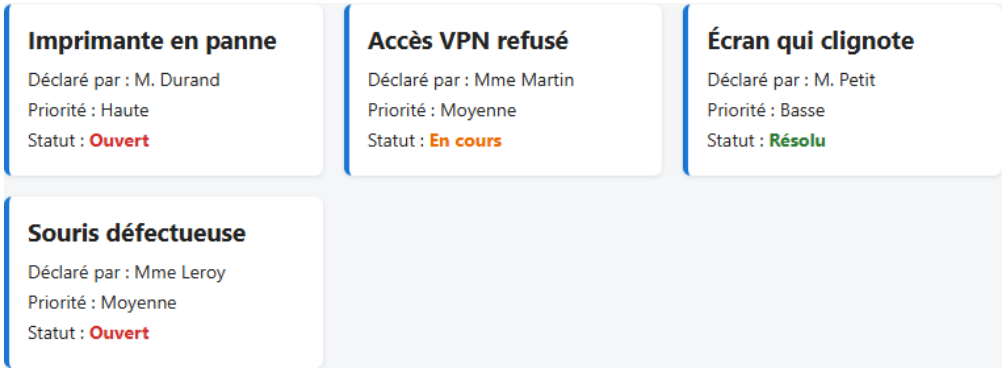
Pour compléter le TP, j'ai ajouté un **champ de recherche** qui filtre les tickets par titre en temps réel (avec **filter()** sur le tableau et un state) :



J'ai aussi ajouté un **formulaire** pour créer un nouveau ticket. Il utilise un **state de tableau** (**setTickets([...tickets, nouveau])**) pour ajouter le ticket saisi à la liste :



Après validation, le nouveau ticket apparaît bien à la fin de la liste :



Enfin, j'ai initialisé un dépôt **Git** et poussé le projet sur GitHub.

Conclusion

Ce TP m'a permis de découvrir les bases de **React** : l'installation de l'environnement avec Node.js et Vite, la structure d'un projet, puis l'écriture de composants en **JSX**, l'utilisation des **props** et du **state** (useState), et l'affichage d'une liste dynamique avec **map** et **key**. J'ai obtenu une petite application de suivi de tickets fonctionnelle, à laquelle j'ai ajouté une recherche et un formulaire de création.

La principale difficulté a été de bien comprendre la différence entre les **props** et le **state**, et de penser à toujours utiliser les fonctions **set…** pour mettre à jour l'affichage plutôt que de modifier directement les variables.